

OSSP

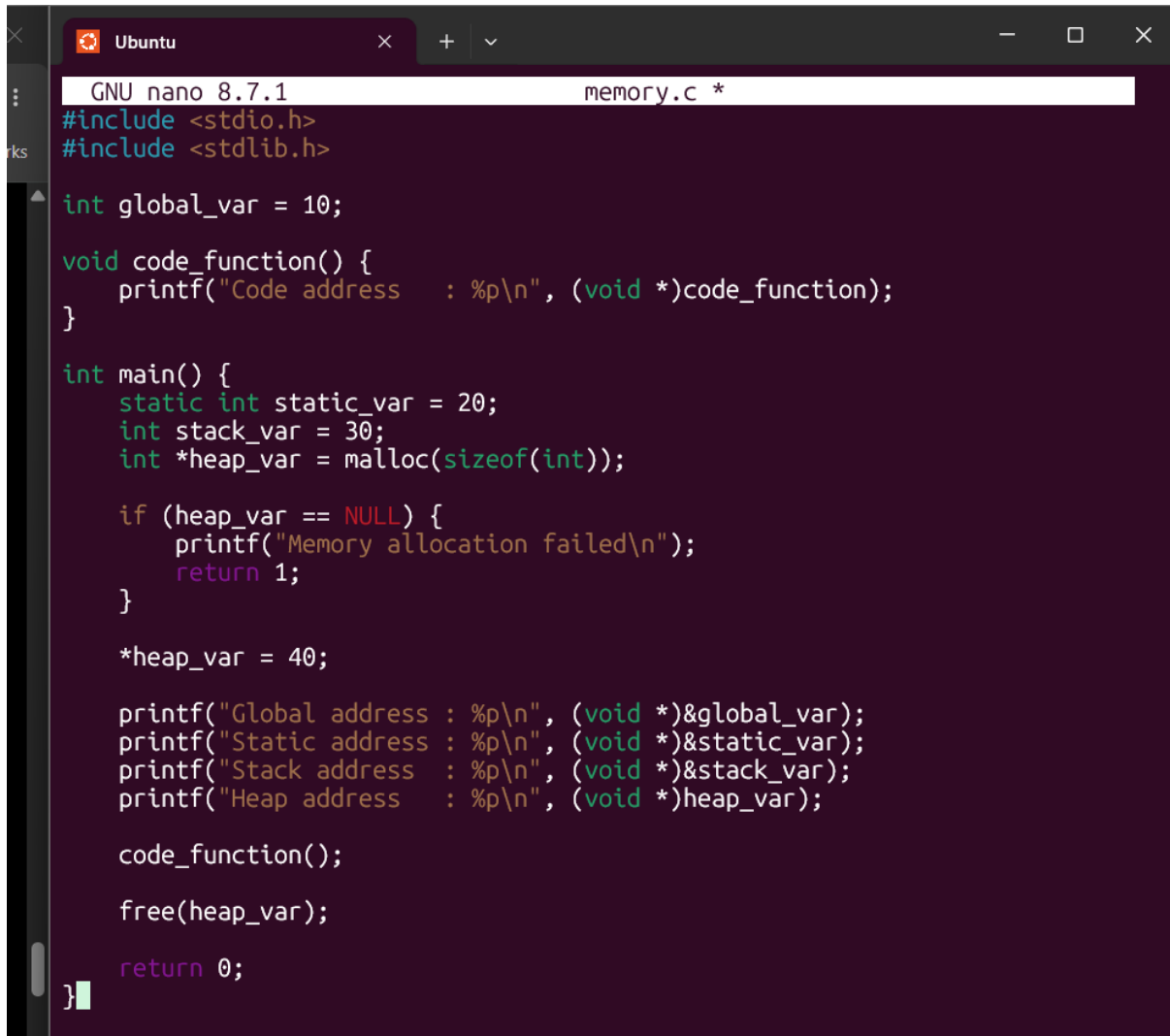
Practical Week -7

Roll No – 2520090085

Name - : Vakkalanka Sai Bhaskara Sujit

Task 1-:

Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

A screenshot of a terminal window with a dark background. The window title is 'Ubuntu'. The editor is 'GNU nano 8.7.1' editing a file named 'memory.c'. The code is a C program that prints the addresses of various memory segments. It includes `<stdio.h>` and `<stdlib.h>`. It defines a global variable `global_var`, a function `code_function`, and a `main` function. In `main`, it declares a static variable `static_var`, a stack variable `stack_var`, and a heap variable `heap_var` (allocated with `malloc`). It checks if `heap_var` is `NULL` and prints an error if so. It then prints the addresses of `global_var`, `static_var`, `stack_var`, and `heap_var` using `printf` with the `%p` format specifier. It also calls `code_function`, frees `heap_var`, and returns 0.

```
GNU nano 8.7.1 memory.c *
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;

void code_function() {
    printf("Code address   : %p\n", (void *)code_function);
}

int main() {
    static int static_var = 20;
    int stack_var = 30;
    int *heap_var = malloc(sizeof(int));

    if (heap_var == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

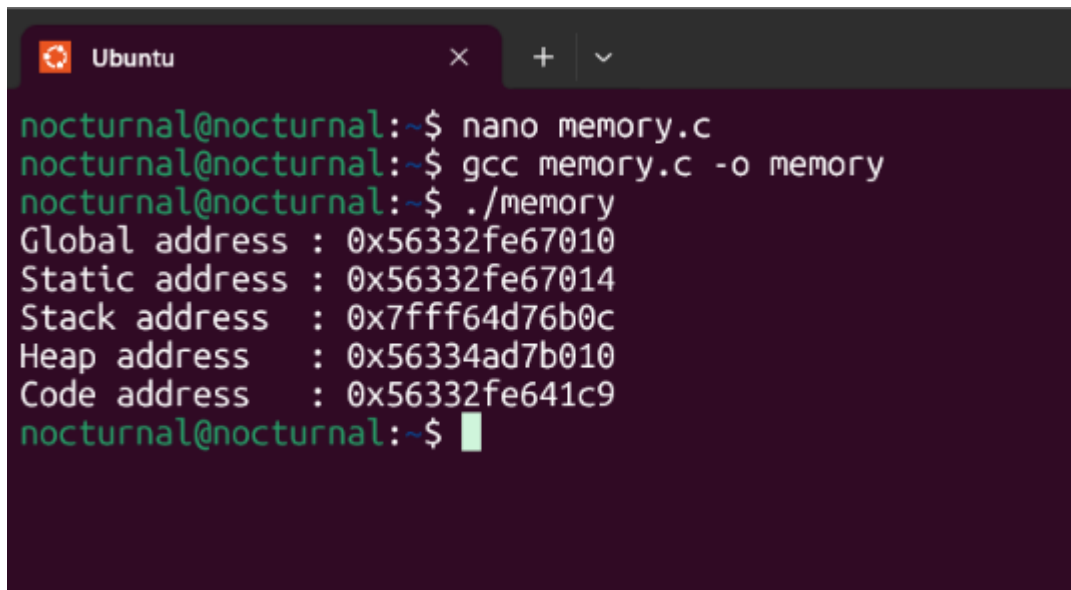
    *heap_var = 40;

    printf("Global address : %p\n", (void *)&global_var);
    printf("Static address : %p\n", (void *)&static_var);
    printf("Stack address  : %p\n", (void *)&stack_var);
    printf("Heap address   : %p\n", (void *)heap_var);

    code_function();

    free(heap_var);

    return 0;
}
```

A terminal window titled 'Ubuntu' with standard window controls (close, maximize, minimize). The terminal shows a user named 'nocturnal' at a host named 'nocturnal' in their home directory. They have executed three commands: 'nano memory.c', 'gcc memory.c -o memory', and './memory'. The output of the third command displays five memory addresses: Global address (0x56332fe67010), Static address (0x56332fe67014), Stack address (0x7fff64d76b0c), Heap address (0x56334ad7b010), and Code address (0x56332fe641c9). The prompt is now ready for the next command.

```
nocturnal@nocturnal:~$ nano memory.c
nocturnal@nocturnal:~$ gcc memory.c -o memory
nocturnal@nocturnal:~$ ./memory
Global address : 0x56332fe67010
Static address : 0x56332fe67014
Stack address  : 0x7fff64d76b0c
Heap address   : 0x56334ad7b010
Code address   : 0x56332fe641c9
nocturnal@nocturnal:~$
```

Task-2:

Make use of the `/proc/<pid>/maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

```
#include <stdio.h>
#include <unistd.h>

int global_var = 100;

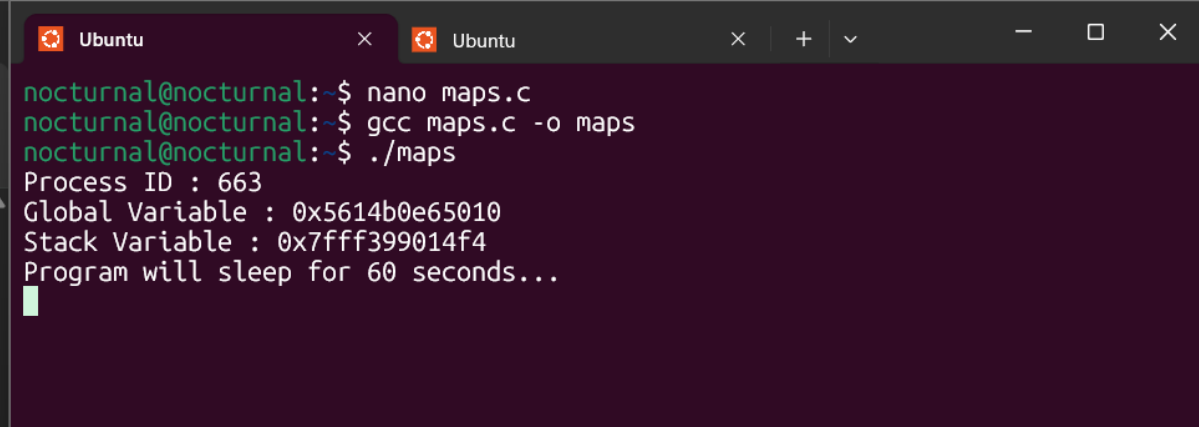
int main() {
    int stack_var = 10;

    printf("Process ID : %d\n", getpid());
    printf("Global Variable : %p\n", &global_var);
    printf("Stack Variable : %p\n", &stack_var);

    printf("Program will sleep for 60 seconds...\n");

    sleep(60);

    return 0;
}
```



The image shows a terminal window with two tabs, both labeled 'Ubuntu'. The terminal displays the following commands and output:

```
nocturnal@nocturnal:~$ nano maps.c
nocturnal@nocturnal:~$ gcc maps.c -o maps
nocturnal@nocturnal:~$ ./maps
Process ID : 663
Global Variable : 0x5614b0e65010
Stack Variable : 0x7fff399014f4
Program will sleep for 60 seconds...
```

The output shows the process ID, the address of the global variable, the address of the stack variable, and a message indicating the program will sleep for 60 seconds.

```
nocturnal@nocturnal:~$ cat /proc/663/maps
5614b0e61000-5614b0e62000 r--p 00000000 08:30 32198 /ho
me/nocturnal/maps
5614b0e62000-5614b0e63000 r-xp 00001000 08:30 32198 /ho
me/nocturnal/maps
5614b0e63000-5614b0e64000 r--p 00002000 08:30 32198 /ho
me/nocturnal/maps
5614b0e64000-5614b0e65000 r--p 00002000 08:30 32198 /ho
me/nocturnal/maps
5614b0e65000-5614b0e66000 rw-p 00003000 08:30 32198 /ho
me/nocturnal/maps
5614ef693000-5614ef6b4000 rw-p 00000000 00:00 0 [he
ap]
73b7dd000000-73b7dd028000 r--p 00000000 08:30 13647 /us
r/lib/x86_64-linux-gnu/libc.so.6
73b7dd028000-73b7dd1c0000 r-xp 00028000 08:30 13647 /us
r/lib/x86_64-linux-gnu/libc.so.6
73b7dd1c0000-73b7dd20e000 r--p 001c0000 08:30 13647 /us
r/lib/x86_64-linux-gnu/libc.so.6
73b7dd20e000-73b7dd212000 r--p 0020d000 08:30 13647 /us
r/lib/x86_64-linux-gnu/libc.so.6
73b7dd212000-73b7dd214000 rw-p 00211000 08:30 13647 /us
r/lib/x86_64-linux-gnu/libc.so.6
73b7dd214000-73b7dd221000 rw-p 00000000 00:00 0
73b7dd40a000-73b7dd40d000 rw-p 00000000 00:00 0
73b7dd413000-73b7dd415000 rw-p 00000000 00:00 0
73b7dd415000-73b7dd419000 r--p 00000000 00:00 0 [vv
ar]
73b7dd419000-73b7dd41b000 r--p 00000000 00:00 0 [vv
ar_vclock]
73b7dd41b000-73b7dd41d000 r-xp 00000000 00:00 0 [vd
so]
73b7dd41d000-73b7dd41e000 r--p 00000000 08:30 13644 /us
r/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
73b7dd41e000-73b7dd44d000 r-xp 00001000 08:30 13644 /us
r/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
73b7dd44d000-73b7dd458000 r--p 00030000 08:30 13644 /us
r/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
73b7dd458000-73b7dd45a000 r--p 0003b000 08:30 13644 /us
```

Procedure: A running process was checked using its process ID. The process ID obtained was 663. The /proc/663/maps file was then opened to view the virtual memory mappings of the process.

Command used: cat /proc/663/maps

Observation: The maps file showed different memory regions with their starting and ending virtual addresses and permissions. It contained mappings for the executable file, shared libraries, heap, stack and some special Linux regions.

Memory Region	Description
Program	Contains the program code and data.

[heap]	Used for dynamically allocated memory.
Shared libraries	Libraries used by the process, such as libc.
Stack	Used for local variables and function calls.
[vvar] / [vdso] / [vsyscall]	Special memory regions provided by Linux.

Permissions: The permission values in the maps file indicate how a memory region can be accessed. r means read, w means write and x means execute.

Result: The virtual memory mappings of the running process were studied successfully using /proc/663/maps.

Conclusion: The task helped me understand that a process has different virtual memory regions for different purposes. The /proc/<pid>/maps file can be used to view these regions and their access permissions.